과제 나만의 문의 응답 체인 (실습)

> 4번째 세션 4-3 체인을 자신의 업무 소재로 변형해 의도한 형식의 응답을 받아봅니다.

성공 기준은 변형한 체인이 의도한 형식으로 응답하는 것입니다.

따라하기

1. 자신의 업무에서 반복되는 문의 유형을 하나 고릅니다. 시스템 운영 담당이라면 장애 신고 접수, 개발 담당이라면 코드 리뷰 요청 접수 같은 소재가 좋습니다.
2. 분류 스키마(Pydantic 모델)를 업무에 맞게 바꿉니다. 부서 대신 장애 등급, 담당 시스템, 처리 유형 등 실제로 쓰는 분류 축을 정의합니다.

```
# 미션 골격 예시: 장애 신고 접수 체인 (소재를 자신의 업무로 교체)
class IncidentReport(BaseModel):
    severity: str = Field(description="장애 등급: 'P1', 'P2', 'P3' 중 하나")
    system: str = Field(description="대상 시스템 이름")
    summary: str = Field(description="장애 현상 한 줄 요약")

# system 프롬프트도 업무 톤으로 교체
# ("system", "너는 장애 접수 담당자야. 접수 확인, 예상 대응, 필요한 추가 정보 요청을 각 한 문장으로 답해.")
```

3. 답변 초안의 system 프롬프트를 업무 톤에 맞게 바꾸고 형식 요구(문장 수와 필수 안내 항목)를 명시합니다.
4. 서로 다른 입력 3개 이상으로 실행해 분류와 형식이 의도대로 나오는지 확인합니다. 일부러 애매한 입력도 넣어 어떻게 분류되는지 봅니다.
5. 완성 코드를 `day1_practice/my_inquiry_chain.py` 로 저장합니다.

막힐 때

- 분류가 의도와 다르게 나옵니다.

  Field의 description을 더 구체적으로 씁니다. 선택지를 명시하고 각 선택지의 기준을 한 줄씩 달아주면 안정됩니다.

- 형식 오류가 납니다.

  오류 메시지 전문을 복사해 3번째 세션 3-3의 막힐 때 항목부터 순서대로 점검합니다.

확인: 입력 3개 이상에서 분류 dict와 답변 초안이 의도한 형식으로 나오면 성공입니다.

> 프롬프트와 스키마를 스스로 바꿔 자신의 체인을 완성하는 경험을 했습니다.

▼ 추가로 해볼 만한 것

- `with_structured_output(Routing)` 으로 4번째 세션 4-3의 분류 체인을 다시 써 봅니다. 프롬프트 형식 지침 없이 같은 결과가 나오는지 비교합니다.
- 4번째 세션 4-3 체인에 5번째 세션 5-3의 `RunnableWithMessageHistory` 를 결합해 이전 문의를 기억하는 상담 체인으로 확장해 봅니다.
- 이 체인의 답변을 `stream` 으로 바꿔 타이핑 효과를 내 봅니다.

▼ 모범답안 (직접 해본 뒤 열어보세요)

장애 신고 접수를 소재로 잡은 완성본입니다. 소재만 다를 뿐 구조는 4번째 세션 4-3 체인과 같습니다.

```
# my_inquiry_chain.py - 장애 신고 접수 체인 (4-3 체인을 운영 업무로 변형)
from dotenv import load_dotenv
from pydantic import BaseModel, Field
from langchain_aws import ChatBedrockConverse
from langchain_core.prompts import PromptTemplate, ChatPromptTemplate
from langchain_core.output_parsers import JsonOutputParser, StrOutputParser
from langchain_core.runnables import RunnableParallel

load_dotenv()
llm = ChatBedrockConverse(
    model="us.anthropic.claude-sonnet-4-5-20250929-v1:0",
    region_name="us-east-1",
    temperature=0,
)


# --- 부품 A: 접수 분류 체인 ---
class IncidentReport(BaseModel):
    severity: str = Field(
        description="장애 등급. P1은 서비스 전면 중단, P2는 일부 기능 장애, P3은 불편 수준.
셋 중 하나만 쓴다"
    )
    system: str = Field(description="대상 시스템 이름. 신고에 없으면 '확인 필요'로 쓴다")
    summary: str = Field(description="장애 현상 한 줄 요약. 30자 이내")


report_parser = JsonOutputParser(pydantic_object=IncidentReport)
report_prompt = PromptTemplate(
    template="다음 장애 신고를 접수 양식으로 정리해.\n\n신고: {inquiry}\n\n{format_instruc
tions}",
    input_variables=["inquiry"],
    partial_variables={"format_instructions": report_parser.get_format_instruct
ions()},
)
report_chain = report_prompt | llm | report_parser

# --- 부품 B: 접수 회신 초안 체인 ---
reply_prompt = ChatPromptTemplate.from_messages([
    ("system",
     "너는 삼성SDS 운영팀 장애 접수 담당자야. "
     "접수 확인, 예상 대응, 신고자에게 요청할 추가 정보를 각각 한 문장으로 써. "
     "총 세 문장을 넘기지 마."),
    ("human", "{inquiry}"),
])
reply_chain = reply_prompt | llm | StrOutputParser()

full_chain = RunnableParallel(report=report_chain, reply=reply_chain)
```

```
if __name__ == "__main__":
    cases = [
        "결제 API가 5분째 502를 뱉습니다. 고객 결제가 전부 실패 중입니다.",    # P1을 기대
        "물류 관제 대시보드에서 어제 자 그래프만 안 보입니다.",                # P2~P3을 기대
        "그거 좀 이상한데 한번 봐주세요.",                                      # 일부러 넣은
애매한 입력
    ]
    for c in cases:
        r = full_chain.invoke({"inquiry": c})
        print("=" * 60)
        print(f"신고: {c}")
        print(f"등급 {r['report']['severity']} / 시스템 {r['report']['system']}")
        print(f"요약: {r['report']['summary']}")
        print(r["reply"])
```

실행 결과는 이렇게 나옵니다.

```
============================================================
신고: 결제 API가 5분째 502를 뱉습니다. 고객 결제가 전부 실패 중입니다.
등급 P1 / 시스템 결제 API
요약: 결제 API 502 오류로 결제 전면 실패
접수 확인했습니다. 결제 API 502 장애를 P1으로 접수했습니다.
운영팀이 게이트웨이 상태와 최근 배포 이력을 바로 확인하겠습니다.
장애 시작 시각과 영향받는 결제 수단을 알려 주시면 원인 범위를 좁힐 수 있습니다.
============================================================
신고: 그거 좀 이상한데 한번 봐주세요.
등급 P3 / 시스템 확인 필요
요약: 대상과 현상이 불명확한 신고
```

설계하면서 걸렸던 부분입니다.

- description에 선택지만 적었을 때는 애매한 입력에서 등급이 실행마다 흔들렸습니다. 각 등급의 판단 기준까지 한 줄로 붙이니 안정됐습니다.
- 값이 없을 때 뭘 채울지("확인 필요")를 스키마에 박아 두면 빈 문자열이나 null이 튀어나오지 않습니다.
- 애매한 입력을 일부러 섞는 게 핵심입니다. 잘 쓴 신고 세 건만 넣으면 스키마가 좋은지 나쁜지 드러나지 않습니다.